

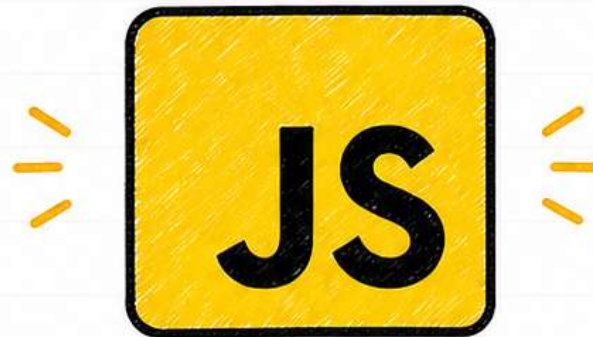


TheAutomationEngineer

JAVASCRIPT

COMPLETE NOTES

(JavaScript)



BEGINNER TO ADVANCED

SWIPE TO NEXT →



FOLLOW



SHARE



COMMENT



LIKE

Achal Singh



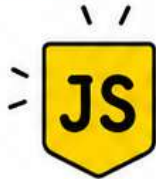
Introduction to JavaScript



① What is JavaScript?

JavaScript (JS) is a lightweight, **interpreted** programming language used to make web pages **interactive** and dynamic.

It is one of the core technologies of the Web, along with **HTML** and **CSS**.



② History

- **1995**: Created by **Brendan Eich** at Netscape.
- Initially named **Mocha**, then **LiveScript**.
- Later renamed to **JavaScript**.
- **ECMAScript (ES)** is the standard specification.
- Regular updates: **ES5** → **ES6 (2015)** → **ES2024** ...

③ Where is JavaScript Used?



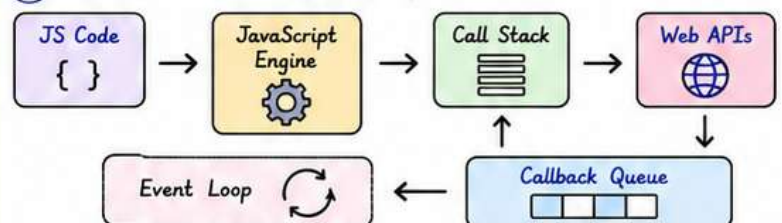
④ Browser vs Node.js

	Browser (Client-side)	Node.js (Server-side)
	Runs inside the web browser.	Runs outside the browser on server.
	Has access to DOM , BOM .	No DOM or BOM by default.
	Single-threaded (event loop).	Single-threaded with event loop.
	More secure (sandboxed).	Full system access (file system, etc.).
	Used for UI / UX and interactivity.	Used for backend, APIs , databases.

⑤ Key Features of JavaScript

- ✓ **Interpreted Language** - No compilation step.
- ✓ **Cross-platform** - Works in all major browsers and on servers (Node.js).
- ✓ **Lightweight** - Small size and fast execution.
- ✓ **Dynamic Typing** - Types are decided at runtime.
- ✓ **First-class Functions** - Functions are treated like any other value.
- ✓ **Event-driven** - Handles user interactions smoothly.
- ✓ **Asynchronous** - Non-blocking with event loop, callbacks, promises, async/await.

⑥ Execution Flow (Browser)



- JS code is executed line by line in the **Call Stack**.
- Long-running tasks are sent to **Web APIs**.
- **Event Loop** checks the **Callback Queue** and pushes the callbacks back to **Call Stack** when it's empty.



Tip

Learn JavaScript in your browser console. Open DevTools (**F12**) → **Console** tab and start experimenting!

Fun Fact

JavaScript was created in just **10 days!**
That's why it's awesome! 😊



Remember

JavaScript = HTML (structure) + CSS (style) + JS (behavior) = **Interactive Web**





Variables & Data Types



① Declaration Keywords

JavaScript has 3 ways to declare variables.

- **var** - Old way (function scoped)
- **let** - Block scoped (reassignable)
- **const** - Block scoped (cannot be reassigned)



Tip:

Always prefer **let** and **const** in modern JavaScript.

Code Examples

```
var x = 10; // function scoped
let y = 20; // block scoped
const z = 30; // cannot be reassigned

y = 25; // ✅ allowed
// z = 40; // ❌ Error: Assignment to constant variable.
```

② Scope Comparison

var (Function Scoped)

```
function test() {
  var a = 10;
  if (true) {
    var a = 20;
  }
  console.log(a); // 20
}
test();
```

VS

let / const (Block Scoped)

```
function test() {
  let b = 10;
  if (true) {
    let b = 20;
  }
  console.log(b); // 10
}
test();
```



var ignores block scope, **let** & **const** respect block scope.

③ Primitive vs Reference Types

Primitive
(Stored by Value)

- String
- Number
- Boolean
- Undefined
- Null
- BigInt
- Symbol

Copied
by value
↔

Reference
(Stored by Reference)

- Object
- Array
- Function
- Date
- RegExp
- Map, Set etc.

Copied
by reference
↔



Changes in primitive **don't** affect original.
Changes in reference **affect** the original.

④ All JavaScript Data Types

Type	Example	typeof	Description
String	"Hello"	"string"	Text data.
Number	42, 3.14	"number"	Integer or floating point.
Boolean	true, false	"boolean"	Logical true or false.
Undefined	undefined	"undefined"	Variable declared but not assigned.
Null	null	"object" (JS historical bug)	Intentional absence of any value.
BigInt	1234567890123456789n	"bigint"	For very large integers.
Symbol	Symbol('id')	"symbol"	Unique and immutable value.
Object	{ name: 'John' }	"object"	Collection of key-value pairs.
Array	[1, 2, 3]	"object"	Ordered list of values.
Function	function() {}	"function"	Reusable block of code.

Check typeof

```
typeof "Hi"; // "string"
typeof 100; // "number"
typeof true; // "boolean"
typeof undefined; // "undefined"
typeof null; // "object"
typeof {}; // "object"
typeof Symbol(); // "symbol"
typeof []; // "object"
typeof function(){}; // "function"
```

⑤ null vs undefined

undefined

- Declared but not assigned
- Default value of variables

null

- Explicitly assigned
- Means "no value"



⑥ BigInt

123

Represents integers larger than
Number.MAX_SAFE_INTEGER
Append **"n"** at the end.

```
let big = 9007199254740991n;
console.log(typeof big); // "bigint"
```

⑦ Symbol

Creates unique identifiers.

Symbols are unique and immutable.

```
let id1 = Symbol('id');
let id2 = Symbol('id');
console.log(id1 === id2); // false
```



Remember: Use **const** by default, **let** when you need to reassign, and avoid **var** in modern JavaScript.



Best Practice: Keep your code clean and use meaningful variable names.



Operators & Type Conversion



① Arithmetic Operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 2	5
%	Modulus	10 % 3	1
**	Exponent	2 ** 3	8
++	Increment	a++	a + 1
--	Decrement	a--	a - 1

② Comparison Operators

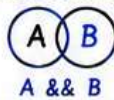
Operator	Name	Example	Result
==	Equal	5 == "5"	true
===	Strict Equal	5 === "5"	false
!=	Not Equal	5 != 3	true
!==	Strict Not Equal	5 !== "5"	true
>	Greater Than	5 > 3	true
<	Less Than	5 < 3	false
>=	Greater or Equal	5 >= 5	true
<=	Less or Equal	4 <= 5	true

③ Logical Operators

Operator	Name	Example	Result
&&	AND	true && false	false
	OR	true false	true
!	NOT	!true	false

Short-circuiting:

- AND (&&) stops if first value is false.
- OR (||) stops if first value is true.



④ Assignment Operators

Operator	Example	Same As
=	x = 10	x = 10
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2
%=	x %= 3	x = x % 3
**=	x **= 2	x = x ** 2

⑤ Ternary Operator

condition ? expressionIfTrue : expressionIfFalse



```
let age = 18;
let result = (age >= 18) ? "Adult" : "Minor";
console.log(result); // Adult
```

⑥ Truthy vs Falsy

Falsy Values
(treated as false)

false
0
0
"" (empty string)
null
undefined
NaN

Truthy Values
(treated as true)

true
1, -1, 123
"Hello"
[] (empty array)
{ } (empty object)
function() { }
"0" (string zero)
Infinity, -Infinity



Tip

In conditionals,
falsy = false
everything else
is truthy = true

⑦ Implicit Type Conversion (Coercion)

JavaScript automatically converts types in some cases.

```
console.log(5 + "5"); // "55" (number + string => string)
console.log("5" - 2); // 3 (string => number)
console.log(true + 1); // 2 (true => 1)
console.log(false == 0); // true (false => 0)
console.log(null == undefined); // true
```

⑧ Explicit Type Conversion (You do it manually)

```
Number("123") // 123
String(456) // "456"
Boolean(0) // false
Boolean(100) // true
Number(true) // 1
String(false) // "false"
```



Remember

- ✓ + - * / % ** are arithmetic.
- ✓ == compares value and type.
- ✓ === compares only value.
- ✓ Use === and !== in real projects.

Type of Operator (typeof)

```
typeof 123; // "number"
typeof "Hi"; // "string"
typeof true; // "boolean"
typeof undefined; // "undefined"
typeof null; // "object" (legacy bug)
typeof Symbol("id"); // "symbol"
typeof function() {}; // "function"
```



Best Practice

- Be careful with == and implicit conversions.
- Write clear, predictable conditions.



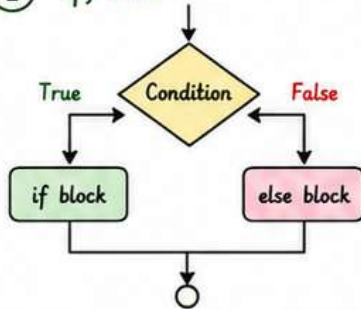


Control Flow & Loops



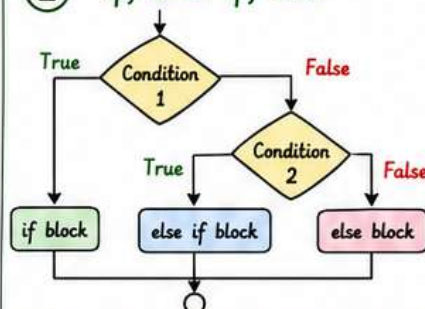
Control flow statements allow your program to make decisions and repeat actions based on conditions.

① if, else



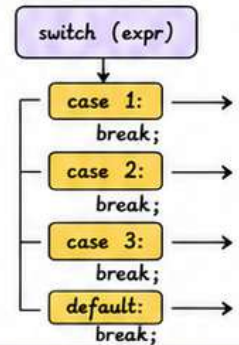
```
let age = 18;
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

② if, else if, else



```
let score = 75;
if (score >= 90) {
  console.log("A");
} else if (score >= 75) {
  console.log("B");
} else {
  console.log("C");
}
```

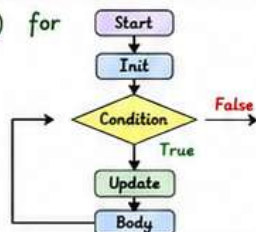
③ switch



```
let day = 2;
switch (day) {
  case 1: console.log("Mon"); break;
  case 2: console.log("Tue"); break;
  case 3: console.log("Wed"); break;
  default: console.log("Invalid");
}
```

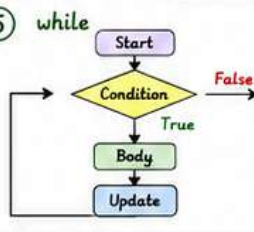
★ LOOPS ★

④ for



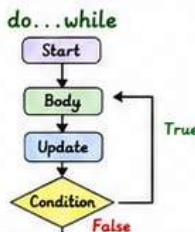
```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
// 1 2 3 4 5
```

⑤ while



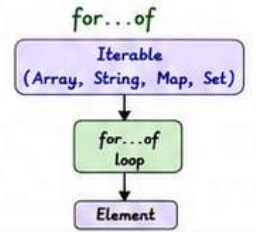
```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
// 1 2 3 4 5
```

⑥ do...while



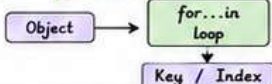
```
let i = 1;
do {
  console.log(i);
  i++;
} while (i <= 5);
// 1 2 3 4 5
```

⑦ for...of



```
let arr = [10, 20, 30];
for (let val of arr) {
  console.log(val);
}
// 10 20 30
```

⑧ for...in



```
let user = { name: "John", age: 25 };
for (let key in user) {
  console.log(key + ": " + user[key]);
}
// name: John // age: 25
```

⑨ break

Terminates the loop or switch immediately.

```
for (let i = 1; i <= 10; i++) {
  if (i == 5) break; // exit loop
  console.log(i);
}
// 1 2 3 4
```

⑩ continue

Skips the current iteration and continues with next iteration.

```
for (let i = 1; i <= 5; i++) {
  if (i == 3) continue; // skip 3
  console.log(i);
}
// 1 2 4 5
```



Tip

- Use break to exit early.
- Use continue to skip unwanted iterations.



Remember

- break & continue work in loops and switch.
- for...in → for objects (keys)
- for...of → for values (iterables)



Best Practice

- Choose the right loop:
for (count) | for...of (values) |
for...in (keys)





Functions



Functions are reusable blocks of code that perform a specific task.

1 Function Declaration

Defined using the function keyword.

```
function add(a, b) {
  return a + b;
}

let sum = add(5, 3); // 8
```

2 Function Expression

Stored in a variable.
Can be anonymous.

```
const multiply = function(a, b) {
  return a * b;
}

let result = multiply(4, 2); // 8
```

3 Arrow Function

Shorter syntax. No own *this* or arguments.

```
const divide = (a, b) => {
  return a / b;
};

const square = x => x * x;
let val = square(5); // 25
```

4 Parameters & Return Values

Parameters are inputs. *Return* sends a value back.

```
function greet(name) {
  return "Hello, " + name;
}

let msg = greet("John");
// Hello, John
```

Input

Function

Output

5 Default Parameters

Provide default values if no argument is passed.

```
function greet(name = "Guest") {
  return "Hello, " + name;
}

greet(); // Hello, Guest
greet("Alice"); // Hello, Alice
```

★ Tip

Defaults make functions more flexible and reduce errors.

6 Rest Parameters (...)

Collects remaining arguments into an array.

```
function sumAll(...nums) {
  return nums.reduce((acc, n) => acc + n, 0);
}

sumAll(1, 2, 3, 4); // 10
sumAll(5, 10); // 15
```

Remember

Rest must be the last parameter.

7 Both Default & Rest Together

```
function createUser(name = "User", ...roles) {
  return { name, roles };
}

createUser("John");
// { name: 'John', roles: [] }
createUser("Jane", "admin", "editor");
// { name: 'Jane', roles: ['admin', 'editor'] }
```

8 Callback Basics

A callback is a function passed as an argument to another function, which is executed later.

```
function process(num, callback) {
  let result = num * 2;
  callback(result); // calling the callback
}

function show(result) {
  console.log("Result is:", result);
}

process(5, show); // Result is: 10
```

Function A
(Higher Order)

accepts

Callback Function

executes later

Real World Analogy



Task



Give task to someone



They complete and notify back

💡 Tip

- Callbacks help in asynchronous operations like in APIs, timers, events, and more.

★ Best Practice

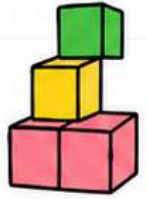
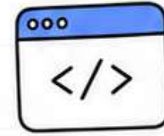
- Use meaningful function names.
- Keep functions small and focused.

- Use default & rest parameters for flexible functions.
- Prefer arrow functions for simple, short logic.





Arrays

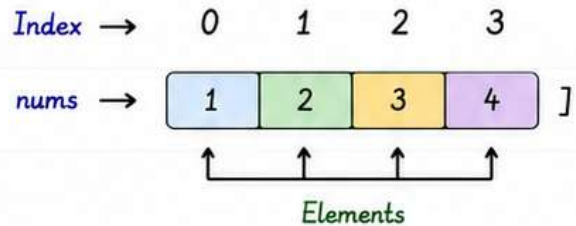


Arrays are ordered collections that can store multiple values in a single variable.

① Creating Arrays

```
const nums = [1, 2, 3, 4];
const fruits = ['Apple', 'Banana', 'Mango'];
const mixed = [1, 'Hello', true, null];
const empty = [];

// Arrays can hold any data type.
```



② CRUD Operations

Create (Add)

- `push()` → add at end
- `unshift()` → add at start

```
let arr = [1, 2];
arr.push(3); // [1, 2, 3]
arr.unshift(0); // [0, 1, 2, 3]
```

Read (Access)

- index starts at 0

```
let arr = [10, 20, 30];
arr[0]; // 10
arr[2]; // 30
arr.length; // 3
```

Update

- change value by index

```
let arr = [10, 20, 30];
arr[1] = 99;
// [10, 99, 30]
```

Delete

- `pop()` → remove last
- `shift()` → remove first

```
let arr = [1, 2, 3];
arr.pop(); // [1, 2]
arr.shift(); // [2]
```

③ Array Methods (Most Used)

map() ★

Creates a new array by applying a function to each element.

```
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);
// [2, 4, 6]
```

filter() ★

Creates a new array with elements that pass the condition.

```
const nums = [1, 2, 3, 4];
const even = nums.filter(n => n % 2 === 0);
// [2, 4]
```

find() ★

Returns the first element that satisfies the condition.

```
const users = [{id:1}, {id:1}, {id:2}];
const user = users.find(u => u.id === 2);
// { id: 2 }
```

reduce() ★

Reduces the array to a single value.

```
const nums = [1, 2, 3, 4];
const sum = nums.reduce((acc, n) => acc + n, 0);
// 10
```

forEach() ★

Executes a function for each element. Does not return a new array.

```
const fruits = ['A', 'B', 'C'];
fruits.forEach(f => console.log(f));
// A B C
```

sort() ★

Sorts the array in place and returns the sorted array.

```
const nums = [3, 1, 4, 2];
nums.sort(); // [1, 2, 3, 4] (as strings)
nums.sort((a, b) => a - b);
// [1, 2, 3, 4] (numeric)
```



Remember

- Arrays are **mutable** (can be changed).
- Index is **zero-based**.
- Methods like **map**, **filter**, **reduce** do not change the original array.



Tip

- Use **map** when you want a new transformed array.
- Use **filter** to select matching items.
- Use **reduce** for calculations.



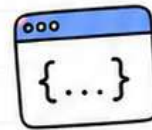
Best Practice

Choose the right method for clean and efficient code.





Objects



Objects are collections of key-value pairs.
Keys are strings and values can be any data type.

① Objects, Properties & Methods

Objects store related data and behaviors (methods).

```
const user = {
  name: 'John',    // property
  age: 25,         // property
  isQA: true,      // property
  greet: function() { // method
    return 'Hello ' + this.name;
  }
};
console.log(user.name); // John
console.log(user.greet()); // Hello John
```

Object

user

```
name : 'John'
age  : 25
isQA : true
greet: function()
```

② Destructuring

Extract values from objects into variables.

```
const user = { name: 'Alice', age: 28, city: 'NY' };

const { name, age } = user;
console.log(name); // Alice
console.log(age);  // 28

// Rename while destructuring
const { city: location } = user;
console.log(location); // NY
```

③ Spread Operator (...)

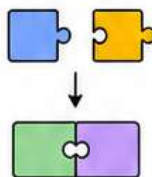
Copies or merges objects.

```
const user = { name: 'John', age: 25 };
const role = { role: 'Tester' };

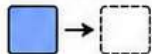
// Merge objects
const emp = { ...user, ...role };
console.log(emp);
// { name: 'John', age: 25, role: 'Tester' }

// Copy object
const copy = { ...user };
copy.age = 30;
console.log(user.age); // 25 (original not changed)
```

Merge



Copy



④ Optional Chaining (?.)

Safely access nested properties.

```
const user = {
  name: 'John',
  address: {
    city: 'Boston'
  }
};
console.log(user.address?.city); // Boston
console.log(user.contact?.phone); // undefined
console.log(user.address?.zip?.code); // undefined
```



Why ?.

Avoids "Cannot read property of undefined" errors.

⑤ Object.keys(obj)

Returns all keys.

```
const user = {
  name: 'John',
  age: 25,
  role: 'QA'
};
console.log(Object.keys(user));
// ['name', 'age', 'role']
```

Object.values(obj)

Returns all values.

```
console.log(Object.values(user));
// ['John', 25, 'QA']
```

Object.entries(obj)

Returns key-value pairs.

```
console.log(Object.entries(user));
// [['name', 'John'],
//  ['age', 25],
//  ['role', 'QA']]
```

hasOwnProperty(key)

Checks if key exists.

```
console.log(user.hasOwnProperty('age')); // true
console.log(user.hasOwnProperty('salary')); // false
```

Object.assign(target, ...sources)

Copy properties to target.

```
const target = { a: 1 };
const source = { b: 2 };
Object.assign(target, source);
console.log(target); // { a: 1, b: 2 }
```

Object.freeze(obj)

Prevents changes to object.

```
const user = { name: 'John' };
Object.freeze(user);
user.name = 'Alice'; // ignored
console.log(user.name); // John
```

Object.seal(obj)

Prevents add / delete, allows update.

```
const user = { name: 'John' };
Object.seal(user);
user.name = 'Mike'; // allowed
delete user.name; // ignored
user.age = 25; // ignored
```



Remember

- Object keys are always **strings**.
- Use **destructuring** for clean and readable code.
- Spread operator creates **shallow copies**.
- Optional chaining improves **code safety**.



Tip

- Use **const** unless you need to reassign.
- Use **freeze** for immutable objects.
- Use **seal** for controlled updates.
- Use **optional chaining** to avoid errors.



Best Practice

- Prefer **const** with objects.
- Use meaningful property names.
- Avoid deep nesting; keep it **simple**.
- Use **object methods** to work efficiently.





Strings & Numbers



Strings represent text. Numbers represent numeric values.
Both are primitive data types in JavaScript.

1 String Methods (Most Used)

Strings are immutable.

<code>length()</code>	→ returns length
<code>toUpperCase()</code>	→ convert to upper case
<code>toLowerCase()</code>	→ convert to lower case
<code>trim()</code>	→ remove spaces
<code>slice(a, b)</code>	→ extract part
<code>substring(a, b)</code>	→ extract part
<code>replace(old, new)</code>	→ replace text
<code>includes(str)</code>	→ check if exists
<code>indexOf(str)</code>	→ first index
<code>split(sep)</code>	→ split to array

```
let text = ' Hello JavaScript';
console.log(text.trim().toUpperCase()); // HELLO JAVASCRIPT
console.log(text.slice(2, 7));          // Hello
console.log(text.includes('Java'));      // true
console.log(text.split(' '));           // [' ', 'Hello', 'JavaScript', '']
```

3 Number Methods

Numbers have useful built-in properties and methods.

```
let num = 123.4567;
console.log(num.toFixed(2));           // '123.46'
console.log(num.toPrecision(4));       // '123.5'
console.log(Number.isInteger(10));     // true
console.log(Number.isNaN('abc'));      // true
```

5 Date Basics

Work with dates and time.

```
let now = new Date();
console.log(now);                     // current date & time
console.log(now.getFullYear());       // 2025
console.log(now.getMonth());          // 0-11 (0 = Jan)
console.log(now.getDate());           // day of month
console.log(now.toLocaleDateString()); // readable date
console.log(now.toLocaleTimeString()); // readable time
```



Remember

- Strings are **immutable**. Methods return a new string.
- Use **template literals** for clean and readable code.
- Use **Number** methods for formatting and validation.
- Use **Math** for calculations and random values.
- Date is an object. Months are **0-indexed** (0 = Jan).



2 Template Literals (ES6)

Use backticks (`) to create strings with embedded expressions.

```
let name = 'John';
let age = 25;
let city = 'New York';

let msg = `Hello ${name},
You are ${age} years old
and live in ${city}.`;

console.log(msg);
```

★ Tip

Backticks allow multi-line strings and easy variable embedding using `\${}`.

Multi-line String

4 Math Object

Used for mathematical constants and functions.

<code>Math.PI</code>	→ value of PI	// 3.141592653589793
<code>Math.abs(x)</code>	→ absolute value	// 6
<code>Math.round(x)</code>	→ round to nearest int	// 6
<code>Math.floor(x)</code>	→ round down	// 5
<code>Math.ceil(x)</code>	→ round up	// 6
<code>Math.max(...vals)</code>	→ largest value	// 20
<code>Math.min(...vals)</code>	→ smallest value	// 5
<code>Math.random()</code>	→ random between 0 and 1	// e.g. 0.3742

```
console.log(Math.PI);
console.log(Math.round(4.6));
console.log(Math.floor(4.6));
console.log(Math.max(10, 20, 5));
console.log(Math.random());
```

★ Example

```
let birth = new Date('2000-05-10T10:30:00');
console.log(birth.getFullYear()); // 2000
console.log(birth.getMonth()+1);  // 5 (May)
console.log(birth.getDate());      // 10
console.log(birth.toString());     // Wed May 10 2000
console.log(birth.toString().slice(0, 8)); // 10:30:00
```



Best Practice

- Always validate and format user input.
- Use **template literals** instead of string concatenation.
- Use **toFixed()** for currency and decimal formatting.
- Use **Math.random()** carefully for fair randomness.
- Always consider timezone when working with dates.



ES6+ Features



Modern JavaScript (ES6 and beyond) makes code cleaner, shorter and more powerful.

① Template Literals (ES6)

Use backticks (`) to create strings with variables and expressions.

```
let name = 'Alice';
let age = 25;
let city = 'Bangalore';
```

```
let msg = `Hello ${name},
You are ${age} years old
and live in ${city}.`;
```

```
console.log(msg);
// Multi-line string
```

★ Benefits

- Multi-line
- Easy to read
- Embed expressions

② Destructuring (ES6)

Extract values from arrays or objects into variables.

Object Destructuring

```
const user = {
  id: 101,
  name: 'John',
  role: 'Tester'
};

const { id, name } = user;
console.log(id); // 101
console.log(name); // John
```

Array Destructuring

```
const fruits = ['Apple',
  'Banana', 'Mango'];

const [first, second] = fruits;

console.log(first); // Apple
console.log(second); // Banana
```

③ Spread & Rest Operator

Spread (...) expands, Rest (...) collects.

Spread (expand)

```
const arr1 = [1, 2];
const arr2 = [3, 4];
const merged = [...arr1, ...arr2];
// [1, 2, 3, 4]
```

```
const obj1 = { a: 1 };
const obj2 = { b: 2 };
const obj = { ...obj1, ...obj2 };
console.log(obj);
// { a: 1, b: 2 }
```

Rest (collect)

```
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}

console.log(sum(1, 2, 3, 4)); // 10
```

```
const [a, b, ...rest] = [1, 2, 3, 4, 5];
console.log(a); // 1
console.log(b); // 2
console.log(rest); // [3, 4, 5]
```

④ Modules (ES6)

Organize code into separate files and reuse.

Export (math.js)

```
export const PI = 3.1416;

export function add(a, b) {
  return a + b;
}

export default function greet() {
  return 'Hello!';
}
```

Import (app.js)

```
import { PI, add } from
  './math.js';

import greet from './math.js';

console.log(PI); // 3.1416
console.log(add(5, 3)); // 8
console.log(greet()); // Hello!
```

• Use **export default** for one main export per file.

⑤ Optional Chaining (?.)

Safely access nested properties.

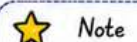
```
const user = {
  name: 'John',
  address: {
    city: 'Hyderabad'
  }
};

console.log(user.address?.city); // Hyderabad
console.log(user.profile?.name); // undefined
console.log(user.address?.zip?.code); // undefined
```



Why?

Avoids "Cannot read property of undefined" errors.



Note

?? works only for null or undefined, NOT for 0, "", false, NaN as empty.



Remember

- Template literals improve readability.
- Destructuring makes code clean.
- Spread copies/merges, Rest collects values.
- Modules help in code reusability.
- Optional chaining prevents runtime errors.
- Nullish coalescing handles null & undefined smartly.



Best Practice

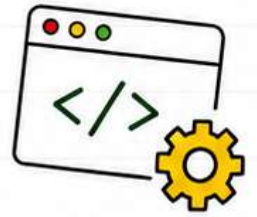
- Use modern ES6+ features for clean code.
- Prefer destructuring over long dot notation.
- Use optional chaining for safe access.
- Use ?? instead of || when needed.
- Always split code into modules.





DOM Manipulation

JavaScript can be used to read, change, add, delete and handle elements in the DOM.



1 Selecting Elements

Use different methods to select elements in the DOM.

- `getElementById()` - by id
- `getElementsByClassName()` - by class
- `getElementsByTagName()` - by tag
- `querySelector()` - first match (CSS)
- `querySelectorAll()` - all matches (CSS)

```
const title = document.getElementById('title');
const items = document.getElementsByClassName('item');
const btn = document.querySelector('button');
const all = document.querySelectorAll('.item');
```

2 Modifying HTML & CSS

Change content, attributes, styles and classes of elements.

- `innerHTML` - change HTML
- `textContent` - change text
- `setAttribute` - set attribute
- `style` - inline styles
- `classList` - add/remove classes

Content

```
const box = document.querySelector('.box');
box.innerHTML = '<b>Hello QA!</b>';
box.textContent = 'Hello QA!';
box.setAttribute('data-id', '123');
box.style.color = 'blue';
box.classList.add('active');
box.classList.remove('hidden');
```

Style & Classes

3 Events & Forms

Respond to user actions and work with form data.

- `click` - element is clicked
- `input` - user types in input
- `submit` - form is submitted
- `change` - value is changed
- `preventDefault()` - prevent default behavior

Form

Name:

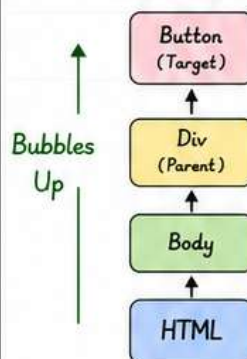
Email:

```
const form = document.querySelector('#myForm');
form.addEventListener('submit', function(e) {
  e.preventDefault(); // stop page reload
  console.log('Form Submitted!');
});
```

4 Event Bubbling & Delegation

Events bubble up from target to ancestors. Delegation uses a single listener on parent.

Event Bubbling



Event Delegation

Attach event to parent and handle events for its children.

```
document.querySelector('#list')
  .addEventListener('click',
    function(e) {
      if (e.target.matches('li')) {
        console.log('Clicked:');
        e.target.textContent;
      }
    });
```



Remember

- ✓ DOM is a tree structure of HTML.
- ✓ Use `querySelector()` for single element.
- ✓ Use `querySelectorAll()` for multiple elements.
- ✓ Prefer `classList` over `className`.
- ✓ Use `preventDefault()` in forms to stop reload.



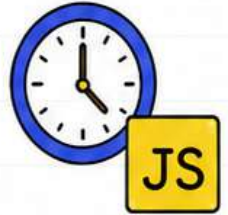
Best Practice

- ✓ Cache DOM elements in variables.
- ✓ Use event delegation for better performance.
- ✓ Avoid excessive DOM updates (reflow/repaint).
- ✓ Keep JS, HTML and CSS separate.
- ✓ Write clean, readable and maintainable code.





Asynchronous JavaScript



JavaScript uses an event-driven, non-blocking model to handle asynchronous operations efficiently.

1 Callbacks

Function passed as an argument.

$f(x)$

2 Promises

Represents the eventual completion (or failure) of an async operation.

Promise

3 Async/Await

Cleaner syntax over Promises. Makes async code look synchronous.

`async () => {}`

4 Fetch API

Modern API to make HTTP requests. (Returns Promise)

`fetch()`

5 Event Loop

Handles async operations using queues (Microtasks & Macrotasks).

1 Callbacks

Oldest way to handle async operations.

```
function getData(cb) {
  setTimeout(() => {
    cb('Data Loaded');
  }, 1000);
}
getData((msg) => console.log(msg));
// Output: Data Loaded
```



2 Promises

More powerful and easier error handling.

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => resolve('Success!'), 1000);
});
promise.then(res => console.log(res))
  .catch(err => console.log(err));
// Output: Success!
```



3 Async / Await

Built on Promises. Easier to write & read.

```
async function loadData() {
  try {
    const res = await fetch('data.json');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log('Error:', err);
  }
}
loadData();
```



4 Fetch API

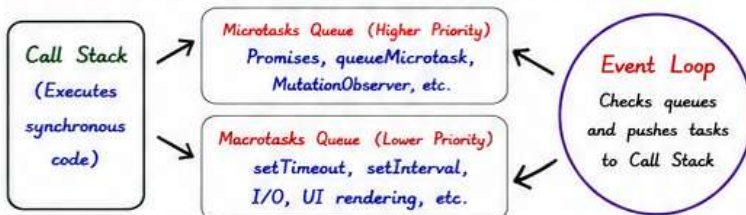
Used to make HTTP requests (returns a Promise).

```
fetch('https://api.example.com/users')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log('Error:', err));
```



5 Event Loop: Microtasks vs Macrotasks

The Event Loop keeps JS single-threaded but non-blocking.



Execution Order

- 1 Run code from Call Stack.
- 2 When stack is empty → Run all **Microtasks**.
- 3 Then pick **ONE Macrotask** from queue and execute.
- 4 Repeat the loop.

Priority

Microtasks (Higher) > Macrotasks (Lower)



Remember

- ✓ Promises & async/await use Microtask Queue.
- ✓ setTimeout & DOM events use Macrotask Queue.
- ✓ Microtasks run before Macrotasks.
- ✓ The Event Loop handles async operations.



Best Practice

- ✓ Use async/await for better readability.
- ✓ Handle errors with try/catch.
- ✓ Avoid blocking the main thread.
- ✓ Understand the Event Loop for better performance.





Error Handling & Debugging

Handle errors gracefully and use debugging tools to find and fix issues faster.



① try, catch, finally

Handle runtime errors using try...catch...
finally block always runs.

```
try {
  let result = riskyFunction();
  console.log(result);
} catch (error) {
  console.log('Error:', error.message);
} finally {
  console.log('Cleanup always runs');
}
```



Use When?

When code may fail and you want to prevent the app from crashing.

② throw & Custom Errors

Use throw to create custom errors.
Create meaningful error messages.

```
function validateAge(age) {
  if (age < 18) {
    throw new Error('Age must be 18+.');
  }
  return 'Valid Age';
}

// Custom Error
class ValidationError extends Error {
  constructor(msg) {
    super(msg);
    this.name = 'ValidationError';
  }
}
```



Tip

Create custom errors for better control and debugging.

③ Console Methods

Use console methods to log and inspect data.

```
console.log('Info message'); // General log
console.info('Some info'); // Info
console.warn('Warning!'); // Warning
console.error('Something went wrong'); // Error
console.table([{id:1},{id:2}]); // Table data
console.assert(2 > 1, 'Not true!'); // Assertion
console.clear(); // Clear console
console.group('User Details'); // Group logs
console.groupEnd(); // End group
```



Why Use?

Helps to log, inspect and understand your code better.

④ Debugging with DevTools

Browser DevTools are your best friend!

File Navigator

Breakpoints

Call Stack & Variables

	Elements	Console	Sources	Network	>>
Page	Filesystem	script.js	Watch		
top		1 function add(a, b) {	result: undefined		
index.html		2 const sum = a + b;			
script.js		3 return sum;	Call Stack		
		4 }	(anonymous)		
		5	script.js:6		
		6 const result = add(2, 3);	Global		
		7 console.log(result);			



Remember

- ✓ Always handle errors, don't ignore them.
- ✓ Use meaningful messages and custom errors.
- ✓ Use console methods to debug effectively.
- ✓ Use breakpoints in DevTools to step through code.
- ✓ Test edge cases and error scenarios.



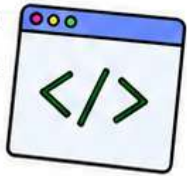
Best Practice

- ✓ Don't expose sensitive error details in production.
- ✓ Log errors for monitoring (e.g. Sentry, LogRocket).
- ✓ Keep try blocks small and specific.
- ✓ Use meaningful variable and function names.
- ✓ Remove console logs before production.





OOP in JavaScript



OOP helps us write clean, reusable and maintainable code using objects and classes.

1 Classes & Constructors

A class is a blueprint for creating objects. Constructor is called when object is created.

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  introduce() {
    return `Hi, I'm ${this.name}`;
  }
}

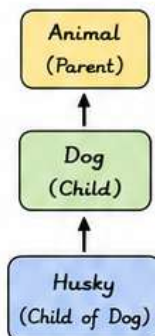
const p1 = new Person('John', 25);
console.log(p1.introduce()); // Hi, I'm John
```



Object is an instance of a class. new keyword creates the object.

2 Inheritance

A class can inherit properties and methods from another class using extends.



```
class Animal {
  speak() {
    return 'Animal speaks';
  }
}

class Dog extends Animal {
  speak() {
    return 'Woof!';
  }
}

class Husky extends Dog {
  speak() {
    return 'Awoo!';
  }
}
```

3 Encapsulation

Bundle data (properties) and methods together. Protect data using private fields (#).

```
class BankAccount {
  #balance = 0; // private property
  constructor(owner) {
    this.owner = owner;
  }
  deposit(amount) {
    if (amount > 0) this.#balance += amount;
  }
  getBalance() {
    return this.#balance;
  }
}
```



Use # to make fields private. They can't be accessed directly.

4 Polymorphism

Same method name behaves differently in different classes.

```
class Animal {
  speak() {
    return 'Some sound';
  }
}

class Cat extends Animal {
  speak() {
    return 'Meow!';
  }
}

class Cow extends Animal {
  speak() {
    return 'Moo!';
  }
}
```

Polymorphism in Action

```
const animals = [
  new Cat(),
  new Cow(),
  new Dog()
];

animals.forEach(a => console.log(a.speak()));
```

5 this Keyword

Refers to the object that is calling the method.

```
class User {
  constructor(name) {
    this.name = name;
  }
  show() {
    console.log(this.name);
  }
}
```



this helps access object properties inside class methods.

6 Prototypes

Every object has an internal link to its prototype.

```
function Person(name) {
  this.name = name;
}

Person.prototype.greet = function() {
  return `Hello ${this.name}`;
};

const p = new Person('Alice');
console.log(p.greet()); // Hello Alice
```



Methods added to prototype are shared by all instances.



Remember

- ✓ Class is a blueprint, object is an instance.
- ✓ Inheritance promotes code reusability.
- ✓ Encapsulation protects data.
- ✓ Polymorphism allows flexibility.
- ✓ Prototypes are the backbone of JS OOP.



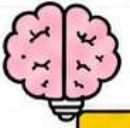
Best Practice

- ✓ Use classes and ES6 features for OOP.
- ✓ Keep properties private when possible.
- ✓ Follow single responsibility principle.
- ✓ Use meaningful names for classes and methods.





Scope, Closures & Hoisting

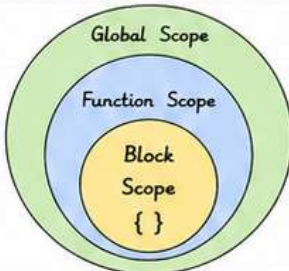


JS

Understanding how JavaScript finds variables, remembers the environment and runs your code.

① Scopes in JavaScript

Variables live in different scopes.
JS looks for a variable in the nearest scope.



- **Global Scope**
Available everywhere in the code.
- **Function Scope**
Available within the function only.
- **Block Scope**
Available within {} (let, const).

```
let globalVar = 'I am global';
function test() {
  let funcVar = 'I am function';
  {
    let blockVar = 'I am block';
  }
}
```

// globalVar → everywhere
// funcVar → in test()
// blockVar → in block only

② Lexical Scope

Inner functions have access to variables from their outer (lexical) scope.

```
function outer() {
  let outerVar = 'I am outer';

  function inner() {
    console.log(outerVar); // Access
  }

  inner();
}

outer(); // Output: I am outer
```



Key Point

JS uses lexical scope (scope is defined by where the function is written, not where it is called).

③ Closures

A closure is a function that remembers variables from its outer scope even after the outer function has finished execution.

```
function counter() {
  let count = 0;
  return function () {
    count++;
    console.log(count);
  }
}

const inc = counter();
inc(); // 1 inc(); // 2 inc(); // 3
```



Why Useful?

- ✓ Data privacy
- ✓ Function factories
- ✓ State maintenance

④ Hoisting

JS moves declarations to the top of their scope before code execution.

Var Hoisted (initialized as undefined)

```
console.log(a); // undefined
var a = 10;
console.log(a); // 10
```

Let & Const (Not hoisted)

```
console.log(b); // ReferenceError
let b = 20;
console.log(b); // 20
```



Rule

var → hoisted & initialized as undefined
let/const → in Temporal Dead Zone (TDZ) until the line is executed.

⑤ Execution Context

JS code runs inside an Execution Context.
There are 2 phases.

Execution Context
Variable Environment (Memory)
Lexical Environment (Scope Chain)
this (Value of this)

① Creation Phase

- Allocate memory for variables and functions.
- Set undefined (var) / TDZ (let/const).

② Execution Phase

- Assign values.
- Execute code line by line.



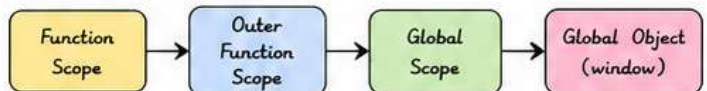
Remember

- ✓ Use let and const for better scope control.
- ✓ Closures help in maintaining state.
- ✓ Hoisting works differently for var vs let/const.
- ✓ Understand execution context to debug better.



⑥ Scope Chain (How JS Resolves Variables)

JS looks for a variable in the current scope, then moves up the chain.



Example: If a variable is not found in the current scope, JS looks in the outer scope and so on until Global scope.



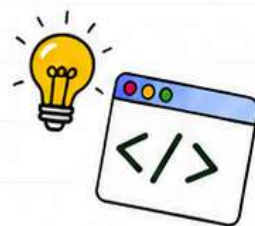
Best Practice

- ✓ Keep scopes small and predictable.
- ✓ Avoid using var.
- ✓ Use meaningful names and minimize globals.
- ✓ Use closures carefully to avoid memory leaks.





Advanced JavaScript



Powerful concepts that make your code more efficient, reusable and elegant.

① Higher-Order Functions

Functions that take other functions as arguments or return a function.

```
function operation(a, b, fn) {
  return fn(a, b);
}

const add = (x, y) => x + y;
console.log(operation(5, 3, add)); // 8
```

Example Use

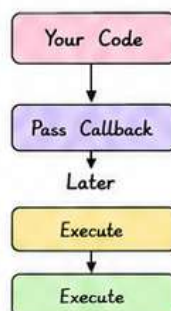
- ✓ map()
- ✓ filter()
- ✓ reduce()
- ✓ forEach()

② Callbacks

A function passed into another function and executed later.

```
function fetchData(callback) {
  setTimeout(() => {
    callback("Data Loaded");
  }, 1000)
}

fetchData((msg) => console.log(msg));
// Output: Data Loaded
```



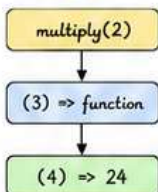
③ Currying

Transform a function with multiple args into a series of functions with a single arg.

```
function multiply(a) {
  return function (b) {
    return function (c) {
      return a * b * c;
    }
  }
}

const result = multiply(2)(3)(4);
console.log(result); // 24
```

Currying Flow



④ Memoization

Store results of expensive function calls and return cached results.

```
function memoize(fn) {
  const cache = {};
  return function (...args) {
    const key = JSON.stringify(args);
    if (cache[key]) return cache[key];
    cache[key] = fn.apply(this, args);
    return cache[key];
  }
}
```



Why Useful?

- ✓ Improves performance
- ✓ Avoids redundant calculations
- ✓ Saves time & resources

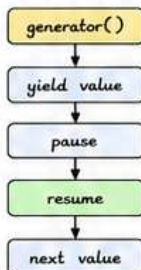
⑤ Generators

Functions that can pause execution and return multiple values over time.

```
function* numberGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = numberGenerator();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3
```

How it Works

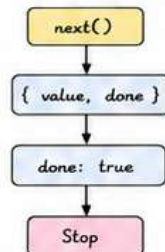


⑥ Iterators

An object that defines a sequence and has a next() method that returns { value, done }.

```
const arr = [10, 20, 30];
const iterator = arr[Symbol.iterator]();
console.log(iterator.next());
// { value: 10, done: false }
console.log(iterator.next());
// { value: 20, done: false }
console.log(iterator.next());
// { value: 30, done: false }
console.log(iterator.next());
// { value: undefined, done: true }
```

Iterator Flow



Remember

- ✓ HOF & Callbacks make code reusable.
- ✓ Currying helps in partial application.
- ✓ Memoization improves performance.
- ✓ Generators & Iterators handle sequences efficiently.



Best Practice

- ✓ Use these concepts to write clean & efficient code.
- ✓ Avoid overusing advanced features.
- ✓ Always consider readability and maintainability.
- ✓ Use memoization only when it's really needed.





JavaScript Interview Questions



High-frequency questions with concise answers.

JS

1 What is JavaScript?

A: JavaScript is a high-level, interpreted programming language used to add interactivity to web pages.



7 What is the difference between null and undefined?

A: undefined means a variable is declared but not assigned. null is an intentional empty value assigned by the developer.

undefined

null

2 What is the difference between var, let and const?

A: var is function-scoped and can be re-declared.
let is block-scoped and can be re-assigned.
const is block-scoped and cannot be re-assigned or re-declared.



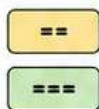
8 What is 'this' keyword?

A: 'this' refers to the object that is currently executing the code. Its value depends on how a function is called.



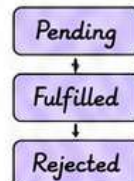
3 What is the difference between == and ===?

A: == checks value only while type coercion.
=== checks both value and type.



9 What is a promise?

A: A Promise is an object representing the eventual completion (or failure) of an asynchronous operation.



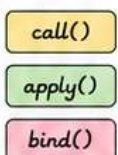
4 What is hoisting?

A: Hoisting is JavaScript's default behavior of moving declarations to the top of their scope before code execution.



10 What is the difference between call, apply and bind?

A: call and apply invoke the function immediately with a given this value. bind returns a new function with the this value set.



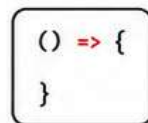
5 What is a closure?

A: A closure is a function that remembers variables from its outer scope even after the outer function has finished executing.



11 What are arrow functions?

A: Arrow functions provide a shorter syntax and do not have their own this, arguments, or prototype.



6 What is the event loop in JavaScript?

A: The event loop allows JavaScript to perform non-blocking operations by handling tasks, callbacks, and promises in queues.



12 What is a prototype?

A: A prototype is an object from which other objects inherit properties and methods.



Remember

- ✓ Understand concepts, not just syntax.
- ✓ Practice code daily and build mini projects.
- ✓ Read error messages carefully.
- ✓ Keep learning and stay consistent.



Pro Tip

- ✓ Explain your thought process in interviews.
- ✓ Write clean, readable and modular code.
- ✓ Practice with real-world scenarios.
- ✓ Stay updated with latest JS features.





JavaScript Best Practices

Write clean, efficient, secure and maintainable JavaScript code.



Aa

1 Clean Code



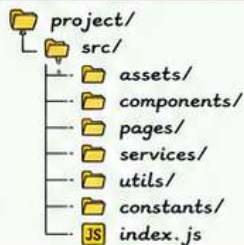
- ✓ Keep functions small and focused.
- ✓ Use meaningful names for variables, functions and classes.
- ✓ Avoid code duplications.
- ✓ Write self-explanatory code & add comments where needed.
- ✓ Follow consistent formatting and indentation.

```
// Good
function calculateTotal(price, tax) {
  const total = price + price * tax;
  return total;
}
```

3 Folder Structure



- ✓ Organize files and folders by feature.
- ✓ Keep components, utils, services and assets separated.
- ✓ Use index.js as the entry point.
- ✓ Avoid deeply nested folders.



Why Useful?

Good structure improves readability, scalability and team collaboration.

5 Security Basics



- ✓ Never trust user input. Validate and sanitize.
- ✓ Avoid using eval() or Function constructor.
- ✓ Use HTTPS for all network requests.
- ✓ Escape data before inserting into DOM.
- ✓ Store sensitive data securely, Avoid localStorage for critical info.



Avoid

- ✗ innerHTML with user input
- ✗ eval(), new Function()



Remember



- ✓ Clean code is easy to read and maintain.
- ✓ Follow consistent patterns across the project.
- ✓ Performance and security are everyone's responsibility.
- ✓ Small improvements today, big impact tomorrow.

2 Naming Conventions

- ✓ Use camelCase for variables and functions.
- ✓ Use PascalCase for constructors, classes.
- ✓ Use UPPER_SNAKE_CASE for constants.
- ✓ Use meaningful and pronounceable names.

Type	Example
Variable / Function	userName, getUserData()
Class / Constructor	UserProfile, UserService
Constant	MAX_RETRY_COUNT
Boolean	isLoggedIn, hasAccess

4 Performance Tips



- ✓ Avoid global variables and unnecessary re-renders.
- ✓ Use const and let instead of var.
- ✓ Debounce or throttle expensive operations.
- ✓ Lazy load modules and images.
- ✓ Use array methods and Set/Map for better performance.

```
// Debouncing Example
const debounce = (fn, delay) => {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
};
```

6 Code Organization



- ✓ Follow Single Responsibility Principle.
- ✓ Group related code together.
- ✓ Use modules and export/import.
- ✓ Keep business logic separate from UI.
- ✓ Write reusable and testable code.

```
// Good: Separate concerns
// userService.js
export const getUser = (id) => { /* API call */ };
// userComponent.js
import { getUser } from './userService.js';
```



Pro Tip

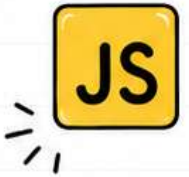
- ✓ Refactor code regularly.
- ✓ Use Linters (ESLint) and formatters (Prettier).
- ✓ Write unit tests for important logic.
- ✓ Keep learning and stay updated.





Ultimate JavaScript

Cheat Sheet



One Page • Quick Reference • Keep it Handy!

1. VARIABLES

- **let** → block scoped
- **const** → block scoped (cannot reassign)
- **var** → function scoped (avoid using)

```
let age = 25;
const name = "QA";
var isAdmin = false;
```

2. OPERATORS

- Arithmetic + - * / %
- Comparison == === != !==
- Logical && || !
- Assignment = += -= *= /=
- Ternary condition ? a : b

```
let max = (a > b) ? a : b;
```

3. FUNCTIONS f(x)

- Function Declaration
- Arrow Function
- Default Parameters
- Rest Parameters
- Return a value

```
const add = (a, b = 0) => {
  return a + b;
};
```

4. ARRAYS

- Ordered collection
- Zero-indexed
- Can store different types
- Has many powerful methods

```
const arr = [1, 2, 3];
arr.push(4); // [1,2,3,4]
arr.map(x => x * 2);
```

5. OBJECTS {}

- Key-value pairs
- Keys are strings (or symbols)
- Access with . or []

```
const user = {
  name: "QA",
  role: "Tester"
};
console.log(user.name);
```

6. ES6+ FEATURES ★

- let, const
- Template literals
- Destructuring
- Spread / Rest Operator
- Classes
- Modules (import / export)

```
const { name } = user;
const nums = [...arr, 5];
export const PI = 3.14;
```

7. ASYNC JAVASCRIPT ⌚

- Promise
- async / await
- fetch() for API calls

```
async function getData() {
  const res = await fetch(url);
  const data = await res.json();
  return data;
}
```

8. DOM MANIPULATION

- Select elements
- Change content / style
- Handle events

```
const el = document.querySelector('.item');
el.textContent = "Hello";
el.style.color = "blue";
el.addEventListener('click', () => {
  console.log('Clicked!');
});
```

9. LOOPS ↻

- for
- while
- do...while
- for...of (arrays)
- for...in (objects)

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

10. STRING METHODS T

- length
- toUpperCase() / toLowerCase()
- trim()
- slice() / substring()
- replace() / split()

```
const msg = ' Hello JS ';
msg.trim().toUpperCase();
msg.replace('JS', 'World');
```

11. COMMON ARRAY METHODS

- map() → transform
- filter() → filter items
- find() → first match
- reduce() → reduce to single value
- some() → any match?
- every() → all match?

```
arr.filter(x => x % 2 === 0);
arr.reduce((a, b) => a + b, 0);
```

12. INTERVIEW QUICK TIPS 💡

- ✓ Understand scope & this
- ✓ Know event loop & async
- ✓ Practice array & string methods
- ✓ Be clear with closures
- ✓ Write clean, readable code
- ✓ Keep learning & building! 



REMEMBER

- ✓ Use const by default.
- ✓ Avoid global variables.
- ✓ Keep functions small & focused.
- ✓ Code → Readability → Maintainability.



PRO TIP

- ✓ Use DevTools for debugging.
- ✓ Prefer === over ==.
- ✓ Test your code in all browsers.
- ✓ Small improvements today, big impact tomorrow!

